

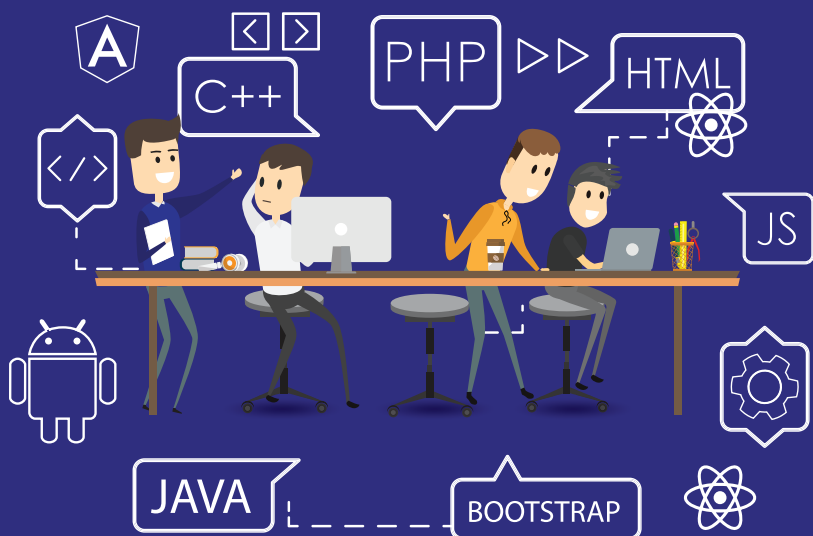
# CODEGYM

Raising the bar

HỆ THỐNG ĐÀO TẠO LẬP TRÌNH HIỆN ĐẠI

# NGHỀ LẬP TRÌNH

TẤT TẦN TẬT NHỮNG ĐIỀU  
NGƯỜI HỌC LẬP TRÌNH KHÔNG THỂ BỎ QUA





Nghề lập trình là nghề có mức lương hấp dẫn, cơ hội thăng tiến và cơ hội việc làm lớn nhất hiện nay.

Mức lương sau 2 - 5 năm kinh nghiệm trung bình từ 13 - 30 triệu tùy năng lực (Theo TOPDev).

Vậy bạn có biết:

- Học lập trình xong ra làm gì? Ở đâu?
- Các mức lương phổ biến lập trình viên nhận được là bao nhiêu?
- Nghề lập trình viên có quan trọng bằng cấp không?
- Ứng viên lập trình nên viết CV như thế nào?
- Làm thế nào để trúng tuyển khi đi phỏng vấn ở các công ty phần mềm?

Mọi thứ sẽ được bật mí trong:

**"Nghề lập trình - Tất tần tật những điều người học lập trình không thể bỏ qua".**

## 1. Nghề lập trình

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1.1. Tổng quan nghề lập trình.....                                | 02 |
| 1.2. Nghề lập trình web - lựa chọn nào cho người mới bắt đầu..... | 03 |
| 1.3. Các vị trí công việc công nghệ thông tin.....                | 06 |
| 1.4. Lập trình viên làm việc ở những đâu?.....                    | 11 |
| 1.5. Lập trình viên nên học ngôn ngữ nào?.....                    | 12 |

## 2. Toàn cảnh ngành CNTT Việt Nam

|                                                  |    |
|--------------------------------------------------|----|
| 2.1. Toàn cảnh ngành CNTT Việt Nam.....          | 14 |
| 2.2. Những con số thống kê trong ngành CNTT..... | 15 |

## 3. Trở thành lập trình viên

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 3.1. Mẫu CV cho lập trình viên.....                                        | 19 |
| 3.2. Các câu hỏi thường gặp khi phỏng vấn<br>tại các công ty phần mềm..... | 20 |
| 3.3. Kỹ năng phỏng vấn.....                                                | 21 |
| 3.4. Nên làm gì để bắt đầu theo đuổi nghề lập trình.....                   | 23 |

## 4. Về CodeGym

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 4.1. Về CodeGym.....                                                                | 26 |
| 4.2. Trở thành lập trình viên chuyên nghiệp<br>sau 4 tháng với Coding Bootcamp..... | 27 |
| 4.3. Lộ trình học tập.....                                                          | 28 |
| 4.4. Coding Bootcamp đầu tiên tại Việt Nam.....                                     | 29 |
| 4.5. Cam kết việc làm.....                                                          | 29 |

## 5. Quick reference

|                                  |    |
|----------------------------------|----|
| 5.1. PHP - Quick reference.....  | 30 |
| 5.2. JAVA - Quick reference..... | 34 |

## 6. Đăng ký tuyển sinh

|                                       |    |
|---------------------------------------|----|
| 6.1. Hướng dẫn đăng ký xét tuyển..... | 44 |
|---------------------------------------|----|

## Tổng quan nghề lập trình

### Nghề lập trình

Để biết nghề lập trình là gì, trước tiên chúng ta phải biết lập trình viên là gì? Lập trình viên là người thiết kế, xây dựng và bảo trì các chương trình máy tính (phần mềm). Bằng cách thao tác các đoạn mã (các ngôn ngữ) trên các công cụ lập trình. Lập trình viên có thể tạo ra các chương trình mới và sửa lỗi hay nâng cấp chương trình đó để tăng tính hiệu quả của việc sử dụng máy tính.

Các lập trình viên thường làm việc trên nhiều ngôn ngữ lập trình như: lập trình web Java, C++, PHP, Asp, ASP.Net và C#.

Vậy, nghề lập trình là nghề thiết kế, xây dựng và bảo trì các chương trình máy tính (phần mềm) bằng cách thao tác các đoạn mã (các ngôn ngữ) trên các công cụ lập trình, nhằm tạo ra các chương trình mới và sửa lỗi hay nâng cấp chương trình đó để tăng tính hiệu quả của việc sử dụng máy tính.

### Lập trình làm công việc gì?

Những người làm nghề lập trình có thể đảm nhận rất nhiều công việc khác nhau. Một trong những công việc chủ yếu của họ chính là viết code. Để làm ra một phần mềm, trước hết người ta phải tạo ra một “bản thiết kế”, mỗi lập trình viên đảm nhiệm một phần việc, sau đó các phần được kết nối lại tạo thành một sản phẩm hoàn chỉnh. Lập trình viên được ví là những thợ “coding” (người ngồi gõ những dòng lệnh (code) trên máy tính), làm ra các phần mềm hoặc chỉnh sửa, phát triển nó dựa trên các công cụ lập trình.

Hiện nay, nghề Lập trình được phân thành nhiều mảng chuyên sâu như: Lập trình Web, Lập trình Hệ Thống, Lập trình Database, Lập trình Game, Lập trình Mobile.

Nhiệm vụ chính của người lập trình là:

- Xây dựng mới một ứng dụng
- Nâng cấp và sửa chữa các ứng dụng có sẵn
- Xây dựng các chức năng xử lý
- Nghiên cứu và phát triển công nghệ mới



## Ngề lập trình web - lựa chọn nào cho người mới bắt đầu

Ngề lập trình Web – lựa chọn nào cho người mới bắt đầu? Full-stack Developer (Lập trình viên Full-stack) là thuật ngữ xuất hiện lần đầu vào năm 2012, từ một thông tin tuyển dụng của Facebook. Kể từ đó, thuật ngữ này đã được sử dụng phổ biến và thảo luận rất nhiều. Thậm chí nó còn trở thành một xu hướng nghề nghiệp rất được quan tâm. Bằng chứng là lượng tìm kiếm của từ khoá “Full-stack Developer” trên toàn thế giới ngày càng tăng theo thời gian.

Vậy Full-stack thực sự có ý nghĩa là gì? Đầu là mối quan hệ giữa các khái niệm Full-stack với Front-end và Back-end?

Nếu là người mới bắt đầu, hẳn bạn cũng rất phân vân về các khái niệm này, phần giải thích sau đây sẽ cho chúng ta thấy các góc nhìn rõ ràng hơn về từng loại công việc và đồng thời mang lại các lời khuyên hữu ích cho những người đang mong muốn có được một sự lựa chọn đúng.

### Nhập môn lập trình web

Hãy bắt đầu bằng việc quay ngược trở lại nguồn gốc của web, để hiểu hơn về cách mà thông tin được hiển thị trên các trang web trên màn hình máy tính của bạn. Tất cả các website đều sử dụng ngôn ngữ HTML để trình bày thông tin theo một cấu trúc mong muốn. Có thể xem HTML chính là phần lõi tạo ra một trang web, như bột để làm bánh vậy – HTML chính là bột và trang web là bánh.

Trong những ngày sơ khai của web, tất cả các trang web đều là tĩnh. Có nghĩa là, nếu lập trình viên muốn thay đổi nội dung của một trang web thì họ cần thay đổi mã HTML của nó thông qua việc cập nhật mã nguồn một cách thủ công. Điều này thật là kinh khủng, bởi vì cứ mỗi lần muốn thay đổi thông tin thì lại phải cập nhật các file HTML.

Thử tưởng tượng, với các website cần cập nhật tin tức hằng ngày, chẳng hạn như BBC News, đội ngũ lập trình viên phải hoạt động vất vả như thế nào để có thể cập nhật được số lượng vài trăm trang web trong một ngày thường xuyên như vậy.

Chắc hẳn, đây không phải là cách làm tốt. Trên thực tế, ngày nay các trang web như của BBC News sử dụng các website động, có nghĩa là nội dung của các trang web có thể dễ dàng cập nhật và thay đổi. Nói một cách dễ hiểu thì đội ngũ lập trình viên đã tạo ra một “bản mẫu” (template) cho các trang tin, sau đó họ sẽ điền các thông tin cụ thể (chẳng hạn như tiêu đề, nội dung, ảnh...) vào các bản mẫu đó. Với cách làm này thì họ không cần phải sửa mã nguồn mỗi khi muốn cập nhật hoặc đăng bài mới nữa.

Vậy các dữ liệu như tiêu đề, nội dung, ảnh... được lưu ở đâu? Đó chính là Cơ sở dữ liệu – một nơi hoàn hảo để lưu trữ và truy xuất các dữ liệu với bất cứ cấu trúc nào mà bạn muốn. Và để làm được điều này, xin mời bạn làm quen với các ngôn ngữ phía máy chủ (server-side language). Có rất nhiều ngôn ngữ làm được việc này, chẳng hạn như Java, C#, PHP, Python, JavaScript...

Và như vậy, ngay bây giờ, một trang web đang hiển thị trên màn hình máy tính của bạn đã có thể trải qua các bước sau:

- Dữ liệu của nó được lưu trữ trong một Cơ sở dữ liệu nào đó, chẳng hạn như MySQL
- Dữ liệu đó được lấy ra và xử lý bởi một ngôn ngữ server-side nào đó, chẳng hạn như Java hoặc PHP
- Mã HTML được sinh ra để hiển thị ngay trên trình duyệt của bạn

### Phát triển ứng dụng Web Front-end

Phát triển ứng dụng Web Front-end là nói đến công việc tạo ra giao diện và trải nghiệm người dùng của một website. Công việc này được thực hiện bởi các lập trình viên Front-end. Thông thường, họ sử dụng các ngôn ngữ như HTML, CSS và JavaScript để hoàn thành công việc của mình. Đây chính là 3 ngôn ngữ tối cần thiết để có thể xây dựng được website trên thực tế.

HTML được dùng để tạo ra nội dung của trang web, CSS giúp cho trang web trở nên đẹp đẽ hơn và JavaScript giúp người dùng tương tác được với trang web một cách tiện lợi.

Nhưng, đây mới chỉ là các công nghệ Web Front-end của thời kỳ đầu. Các lập trình viên Web Front-end ngày nay có những công cụ mạnh mẽ hơn rất nhiều để hoàn thành xuất sắc nhiệm vụ của mình, đó chính là các ngôn ngữ và nền tảng mới, chẳng hạn như TypeScript, CoffeeScript, SASS, LESS, Angular, ReactJS, VueJS...

### Phát triển ứng dụng Web Back-end

Nếu Front-end là bộ mặt của một website thì Back-end chính là bệ đỡ, là nguồn sống của website đó. Phát triển ứng dụng Web Back-end là nói đến công việc sử dụng các ngôn ngữ server-side để thao tác với Cơ sở dữ liệu, thiết lập các quy tắc nghiệp vụ, đảm bảo bảo mật... và cung cấp nội dung cho phía Front-end.

Với tầm quan trọng như vậy, các lập trình viên Web Back-end cần làm việc tốt với nhiều mảng khác nhau, chẳng hạn như cơ sở dữ liệu, bảo mật, các giao thức web, thiết kế kiến trúc hệ thống... và thậm chí là phải biết Front-end.



## Lập trình viên Full-stack

Lập trình viên Web Full-stack là lập trình viên có khả năng làm việc với cả Web Front-end và Web Back-end.

Tại sao lại cần có những lập trình viên Web Full-stack? Bởi vì, mặc dù chúng ta có thể phân chia việc lập trình web thành 2 mảng là Front-end và Back-end, nhưng 2 mảng này đều liên quan rất chặt chẽ với nhau và có ảnh hưởng đến nhau. Đặc biệt, theo xu hướng phát triển hiện nay của mảng Web Front-end thì càng ngày các lập trình viên Front-end càng cần có những kỹ năng gần giống với kỹ năng của lập trình viên Back-end. Có nghĩa là, lập trình viên Web Front-end cũng cần thành thạo thiết kế kiến trúc, thành thạo các giao thức web, thành thạo bảo mật... và thậm chí là biết các kiến thức phía Back-end.

Ngoài ra, một lập trình viên Web Full-stack có rất nhiều thuận lợi khi phát triển các ứng dụng web, so với các lập trình viên chỉ chuyên biệt một mảng là Front-end hoặc Back-end. Lập trình viên Web Full-stack hiểu rõ hơn về tổng quan của hệ thống, biết các thành phần được liên kết với nhau như thế nào, biết cách để các bên giao tiếp với nhau tốt nhất, biết cách để tối ưu toàn bộ hệ thống... và nhất là hiểu được công việc của các bên còn lại.

Trong thực tế, vẫn có nhiều tranh cãi xung quanh thuật ngữ "Lập trình viên Full-stack". Lí do chính là bởi vì có nhiều định nghĩa khác nhau và cách hiểu khác nhau về "Full-stack". Trong khuôn khổ của bài viết này, chúng ta cũng không đặt tham vọng thống nhất được các định nghĩa và cách hiểu. Ngược lại, chúng ta sẽ phân tích ngữ nghĩa để hiểu hơn về một loại công việc mới, thay vì tập trung vào các tranh cãi.

Stack – có nghĩa là một tập hợp các công nghệ khác nhau được sử dụng để phát triển và vận hành một sản phẩm. Chẳng hạn, MySQL – PHP – Linux – Apache – Laravel – Docker – Angular có thể được xem là một stack, tất nhiên là có thể thêm nhiều công nghệ khác vào trong stack này, tùy thuộc vào nhu cầu của từng sản phẩm. MongoDB – Java – Spring MVC – Tomcat – Linux – ReactJS – Docker – AWS cũng có thể là một stack, và cũng có thể thêm nhiều công nghệ khác vào nữa. Các công nghệ này cũng còn được gọi là các tầng (layer) trong một stack. Như vậy, một lập trình viên Full-stack là một lập trình viên có thể làm việc với tất cả các layer của một stack cụ thể. Không có một lập trình viên Full-stack chung chung, mà cần phải chỉ rõ là "stack" nào (stack đó bao gồm những layer nào).

Vậy còn mức độ thuần thục của các công nghệ này thì như thế nào? Tất nhiên, nếu một lập trình viên thành thạo được tất cả các công nghệ trong một stack là điều quá tốt, nhưng điều này thường chỉ đạt được với những lập trình viên khá nhiều năm kinh nghiệm làm việc trong cùng một stack. Đối với các lập trình viên ít kinh nghiệm hơn, thông thường họ sẽ có thể mạnh hơn ở một số công nghệ nào đó, còn các công nghệ còn lại thì họ đủ để dùng để tạo ra sản phẩm. Khi gắn bó với một stack đủ lâu thì mức độ thành thạo của các công nghệ cũng tăng dần lên theo thời gian và thậm chí là lập trình viên đó có thể mở rộng thêm các công nghệ của một stack hoặc bắt đầu làm việc trên các stack khác.





## Chuyên gia phân tích nghiệp vụ (*Business analyst*)

Còn được biết đến với các tên gọi: Kiến trúc sư nghiệp vụ, chuyên gia thông tin.

**Mô tả:** Chuyên gia phân tích nghiệp vụ là người đóng vai trò trung gian quan trọng, làm việc với cả đội ngũ kỹ thuật, các cấp quản lý và với người dùng cuối. Chuyên gia phân tích nghiệp vụ là người đưa ra các cải tiến về quy trình và hoạt động nghiệp vụ thông qua việc sử dụng các công nghệ kỹ thuật. Vai trò này được xác định theo từng dự án, bắt đầu bằng việc phân tích các nhu cầu của khách hàng, thu thập và tài liệu hoá các yêu cầu, lập kế hoạch để xây dựng thiết kế cho giải pháp công nghệ. Chuyên gia phân tích nghiệp vụ cần phải có hiểu biết về công nghệ, tuy nhiên không nhất thiết phải là một chuyên gia công nghệ.

**Các kỹ năng cần thiết:** Giao tiếp, thuyết trình, hỗ trợ, quản lý dự án và giải quyết vấn đề.



## Hỗ trợ kỹ thuật (*Technical support*)

Còn được biết đến với các tên gọi: Nhân viên hỗ trợ, quản lý vấn đề.

**Mô tả:** Hỗ trợ kỹ thuật là người giải quyết các vấn đề trong quá trình hoạt động của các hệ thống. Nhiều chuyên gia hỗ trợ kỹ thuật làm việc trong các công ty sản xuất và cung cấp phần cứng, và cũng có nhiều chuyên gia hỗ trợ kỹ thuật ở các doanh nghiệp nhằm hỗ trợ, theo dõi và bảo trì các hệ thống được sử dụng hằng ngày. Nhiều công việc đòi hỏi các chuyên gia với nền tảng và kinh nghiệm kỹ thuật tốt.

**Các kỹ năng cần thiết:** Hiểu biết nhiều về công nghệ, giải quyết vấn đề, giao tiếp, kiên nhẫn và nhẹ nhàng.



## Kỹ sư mạng (Network engineer)

Còn biết đến với tên gọi: Kỹ sư phần cứng, chuyên gia mạng.

**Mô tả:** Kỹ sư mạng là một trong các công việc rất cần thiết trong ngành IT, là người thực hiện các thao tác cài đặt, quản trị, duy trì và nâng cấp các hệ thống giao tiếp, xử lý các vấn đề liên quan đến mạng lưới trong các công ty. Kỹ sư mạng cũng là người chịu trách nhiệm về bảo mật, lưu trữ dữ liệu và các chiến lược khôi phục nếu có sự cố xảy ra.

**Các kỹ năng cần thiết:** Các kiến thức chuyên ngành mạng, kỹ năng giao tiếp, lập kế hoạch, phân tích và giải quyết vấn đề.



## Quản lý dự án (Project manager)

Còn biết đến với tên gọi: Trưởng dự án.

**Mô tả:** Quản lý dự án là người tổ chức các nhóm phát triển, phân bổ thời gian và tài nguyên để đảm bảo các dự án đạt được các yêu cầu về chức năng, đúng thời gian và nằm trong ngân sách cho phép. Quản lý dự án điều phối tất cả các hoạt động từ khi mới bắt đầu dự án cho đến khi kết thúc. Vai trò này đòi hỏi kinh nghiệm và nền tảng vững chắc về kỹ thuật cũng như các kỹ năng mềm để làm việc tốt với các nhóm phát triển và các nhà quản lý cấp cao.

**Các kỹ năng cần thiết:** Khả năng tổ chức, giải quyết vấn đề, giao tiếp, tư duy rành mạch, khả năng chịu đựng áp lực công việc.



## Nhà phát triển (Developer)

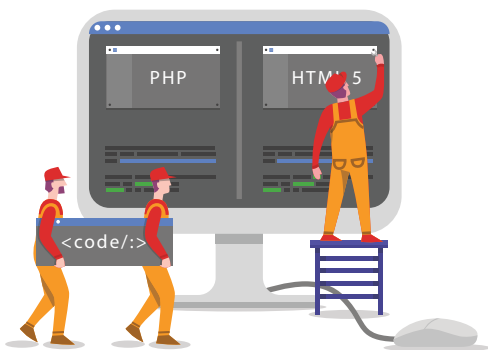


Còn được biết đến với các tên gọi: Lập trình viên, nhân viên viết mã.

**Mô tả:** Nhà phát triển là người trực tiếp tạo ra phần mềm thông qua việc sử dụng các ngôn ngữ lập trình và các công cụ hỗ trợ. Nhà phát triển thường làm việc cộng tác trong các nhóm để đảm bảo các tính năng của sản phẩm được xây dựng và đáp ứng được các yêu cầu như thiết kế ban đầu. Vai trò này đòi hỏi kiến thức và kỹ năng tốt về công nghệ, công cụ và các ngôn ngữ lập trình.

**Các kỹ năng cần thiết:** Hiểu biết về các loại ứng dụng phần mềm, hiểu biết về các ngôn ngữ lập trình, kỹ năng giao tiếp và làm việc nhóm.

## Kiểm thử viên phần mềm (Software tester)



Còn được biết đến với tên gọi: Kiểm thử viên, nhân viên kiểm thử.

**Mô tả:** Kiểm thử viên phần mềm là người tham gia vào công tác đảm bảo chất lượng phần mềm thông qua việc phát hiện các lỗi tiềm tàng và hỗ trợ nhóm phát triển trong việc xử lý các lỗi. Kiểm thử viên phần mềm thực hiện các thao tác phân tích nghiệp vụ, lập kế hoạch kiểm thử, viết các kịch bản kiểm thử, thực thi các ca kiểm thử và viết các báo cáo kiểm thử.

**Các kỹ năng cần thiết:** Tỉ mỉ trong từng chi tiết, khả năng nắm bắt các yêu cầu nghiệp vụ, khả năng sử dụng các hệ thống phần mềm, kỹ năng phân tích và điều tra vấn đề, kỹ năng giao tiếp.

## Chủ sản phẩm (Product owner)

Còn được biết đến với các tên gọi: Quản lý sản phẩm.

**Mô tả:** Chủ sản phẩm là người chịu trách nhiệm xác định và đảm bảo các chức năng của hệ thống. Chủ sản phẩm tìm hiểu và nghiên cứu các yêu cầu của người dùng cuối, đưa ra các giải pháp để đáp ứng được các yêu cầu, quản lý tiến độ và chất lượng của các chức năng trong suốt quá trình phát triển. Chủ sản phẩm là người có hiểu biết về thị trường, về các hệ thống phần mềm và sử dụng các công cụ quản lý sản phẩm và quản lý dự án.

**Các kỹ năng cần thiết:** Tìm kiếm và phân tích thông tin, kỹ năng quản lý, kỹ năng giao tiếp.

## ScrumMaster (ScrumMaster)

**Mô tả:** ScrumMaster là người chịu trách nhiệm trong việc đảm bảo các nhóm scrum hoạt động tốt và chuyển giao được sản phẩm chất lượng cao. ScrumMaster nắm rõ khung làm việc scrum, các kỹ thuật phát triển và là người làm việc thường xuyên với nhóm phát triển. ScrumMaster có thể là người am hiểu về công nghệ hoặc không.

**Các kỹ năng cần thiết:** Kỹ năng giao tiếp, kỹ năng lắng nghe, kỹ năng giải quyết vấn đề, khả năng tháo gỡ các khó khăn.



## Lập trình viên làm việc ở những đâu?

### Công ty gia công phần mềm cho nước ngoài

Có lẽ lợi thế lớn của các công ty gia công nằm ở môi trường chuyên nghiệp và cơ hội tham gia cọ xát các dự án lớn của nước ngoài. Những điều này đặc biệt hữu ích cho những lập trình viên trẻ mới bước từ môi trường giảng đường sang thực tế, và việc đặt mình dưới áp lực của các dự án lớn đòi hỏi các quy trình chặt chẽ sẽ giúp lập trình viên nhanh chóng chuyên nghiệp hóa bản thân mình.

Tuy nhiên môi trường gia công cũng có những hạn chế nhất định, việc vẫn còn ít những dự án xứng tầm cũng như việc trả chi phí cố định theo đầu người ở đa phần các công ty sẽ dẫn khiến nhiều lập trình viên chán nản và từng bước tìm cho mình những hướng đi mới mẻ hơn.

### Công ty start-up/hoặc công ty làm sản phẩm

Làm việc trong một công ty start-up thực sự sẽ đòi hỏi bạn làm việc >100% khả năng của mình. Ngoài ra, do công việc của công ty start-up khá nhiều mà số người lại hạn chế, nên gần như sẽ đòi hỏi bạn phải biết thêm rất nhiều thứ không nằm trong kiến thức của bạn được trang bị ở các công ty lớn. Trong một môi trường như vậy sẽ khiến bạn trưởng thành lên rất nhiều.

Lẽ dĩ nhiên môi trường start-up cũng có những hạn chế. Hạn chế thứ nhất là tỉ lệ các công ty start-up thành công là rất thấp, nên đôi lúc việc gắn bó trong một môi trường start-up chờ thời quá lâu mà vẫn chưa thấy đầu ra sẽ khiến bạn lãng phí rất nhiều thời gian và đánh mất nhiều cơ hội, thực tế chứng minh các công ty start-up thành công lớn đa phần đều có những bước tiến thần tốc trong một thời gian nhất định. Hạn chế thứ hai, môi trường start-up đòi hỏi bạn biết rộng hơn là biết sâu. Do đó nếu bạn là một người chuyên sâu về một công nghệ nhất định thì sẽ không thích hợp trong môi trường này.

### Công ty đa quốc gia

Có lẽ vị trí cho lập trình viên trong các công ty này không nhiều khi phần lớn hiện nay vẫn xem Việt Nam là một thị trường phân phối và tiêu thụ. Tuy nhiên gần đây tình hình đã dần khá hơn khi thị trường Việt Nam bắt đầu lớn và chứng tỏ vị trí tiềm năng trong tương lai gần sắp tới.

Làm việc trong các công ty này được lợi là quy trình đã khá bài bản nên lập trình viên chỉ cần tập trung vào những hướng nhất định, rất phù hợp cho những ai đam mê đào sâu vào một công nghệ mũi nhọn. Ngoài ra làm việc trong môi trường này cũng giúp lập trình viên mở rộng tầm nhìn lên rất nhiều bởi có cơ hội cọ xát trong một môi trường mang tính quốc tế. Tuy nhiên điểm yếu ở các công ty này là lập trình viên sẽ không có được một cái nhìn tổng quát về công ty như trong môi trường công ty start-up đã nêu trên.

### Công ty/tổ chức nhà nước

Làm trong nhà nước thì lương khá thấp so với thị trường. Tuy nhiên đổi lại môi trường tương đối nhẹ hơn, cũng như sẽ có những giá trị vô hình nhất định và tiềm năng trong tương lai, đặc biệt trong một xã hội mà nhà nước vẫn còn nắm giữ một vai trò quan trọng trong nền kinh tế như Việt Nam hiện nay.



## Python

Được phát triển từ những năm 80s bởi Guido Van Rossum, Python cũng nằm trong danh sách ngôn ngữ lập trình phổ biến và dễ tiếp cận. Python là ngôn ngữ mã nguồn mở và sử dụng miễn phí, thậm chí cho các ứng dụng thương mại. Theo đó, Python cho phép các lập trình viên tạo ra một số lượng lớn code dễ đọc trong một khoảng thời gian ngắn. Nhưng Python cũng là một ngôn ngữ Dynamic, hỗ trợ hướng đối tượng và có phong cách lập trình chức năng như những ngôn ngữ khác. Bởi tính mềm dẻo, Python là một trong những ngôn ngữ lập trình bậc cao được sử dụng rộng rãi nhất ngày nay.

## C và C++

C và C++ là khá quen thuộc với hầu hết các sinh viên trong ngành công nghệ thông tin. Bởi, hai ngôn ngữ này thường được đưa vào chương trình giảng dạy ngay từ những năm đầu. C/ C++ được mệnh danh là nền tảng của khoa học máy tính và lập trình. Tuy nhiên, C và C++ được đánh giá là rất khó để học. Dù vậy, nếu nắm được chắc hai ngôn ngữ này, bạn có khả năng học sang ngôn ngữ khác rất nhanh bởi hầu hết các ngôn ngữ đều được phát triển từ một hay một vài khái niệm của C/ C++.



## Toàn cảnh ngành CNTT Việt Nam



**24.501 DOANH NGHIỆP**  
đang hoạt động trong ngành CNTT



**13,13% TĂNG TRƯỞNG**  
số doanh nghiệp CNTT



**1.500.009 TỶ ĐỒNG**  
doanh thu toàn ngành



**34.320 TỶ ĐỒNG**  
nộp ngân sách nhà nước



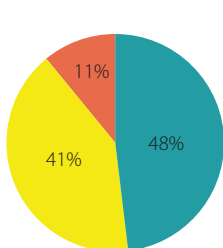
**60 TỶ USD**  
tổng giá trị xuất khẩu



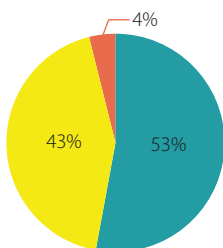
**780.926 NHÂN LỰC**  
đang làm việc trong ngành CNTT

## Nhu cầu tuyển dụng nhân lực CNTT

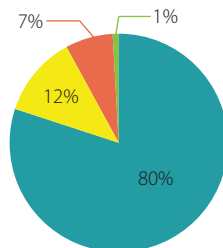
Theo báo cáo của bộ Thông tin và Truyền thông năm 2017, từ nay đến 2020 Việt Nam thiếu 500.000 nhân lực cho ngành CNTT.



Thành phố Hồ Chí Minh và Hà Nội là hai địa phương có nhu cầu tuyển dụng nhân lực CNTT lớn nhất nước



Các công ty gia công là những đơn vị có nhu cầu tuyển dụng nhân lực CNTT cao nhất



80% các vị trí tuyển dụng yêu cầu có hơn 2 năm kinh nghiệm làm việc

■ Gia công ■ Khác  
■ Sản phẩm

■ Hồ Chí Minh ■ Khác  
■ Hà Nội

■ Có kinh nghiệm ■ Quản lý  
■ Mới tốt nghiệp ■ Khác



## THỬ THÁCH

**Sự trung thành và gắn bó với công ty**

Hiện tượng nhảy việc xảy ra thường xuyên, gây khó khăn và tổn thất cho các doanh nghiệp trong việc đào tạo nhân lực và đảm bảo nguồn lực cho các dự án.

**Mức độ trưởng thành và văn hóa làm việc**

Nhân sự không quan tâm đến việc quan sát và củng cố các kinh nghiệm có được từ môi trường làm việc.

**Chất lượng nhân sự thấp**

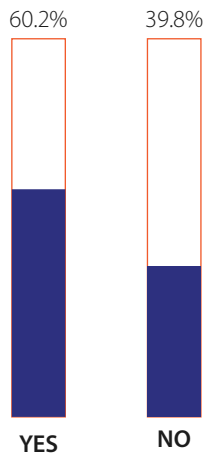
Nhân sự thiếu kiến thức, kỹ năng và kinh nghiệm để đảm nhận các vị trí trong các dự án phần mềm.

**Ảnh hưởng của cách mạng công nghiệp 4.0 và phong trào khởi nghiệp**

Nhiều doanh nghiệp công nghệ mới được hình thành khiến cho thị trường tuyển dụng trở nên sôi động, làm cho mức lương tăng cao nhanh chóng, tạo nên bong bóng lương trong ngành IT.

**Khả năng giao tiếp bằng ngoại ngữ**

Nhiều khách hàng từ châu Âu, Châu Mỹ, Nhật Bản, Hàn Quốc đòi hỏi nhân sự phải giao tiếp được bằng ngoại ngữ, đây là một yêu cầu khó khăn đối với nhân sự IT hiện tại.

**Những con số thống kê trong ngành CNTT****Tỷ lệ nhảy việc cao**

Hơn 60% nhân sự CNTT mong muốn chuyển sang làm việc ở các công ty khác

# TOÀN CẢNH NGÀNH CNTT VIỆT NAM CODEGYM

## Các lý do chính để chuyển việc

47.5%

Mong muốn lương cao hơn

17.5%

Mong muốn thăng tiến

10%

Mong muốn môi trường làm việc tốt hơn

## Các động lực chính để gắn bó với ngành CNTT



62%

Cơ hội thăng tiến



71%

Tự chủ tài chính

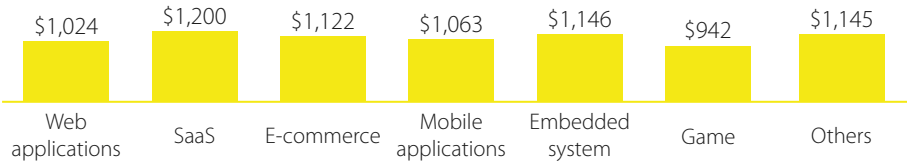


83%

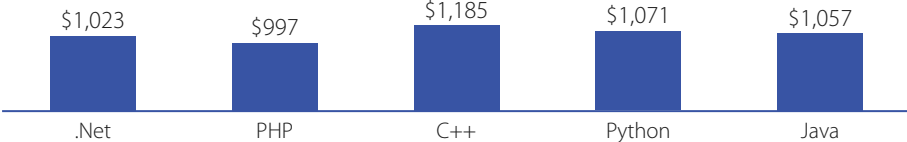
Nâng cao năng lực

## Phân bố lương trong ngành CNTT

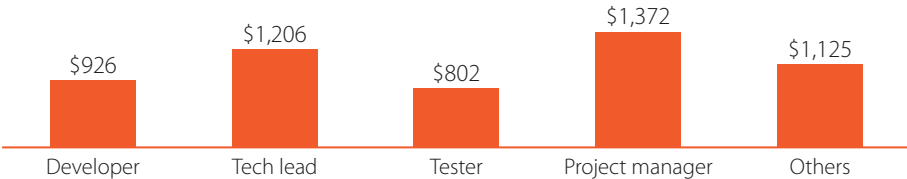
### Theo loại sản phẩm



### Theo ngôn ngữ lập trình



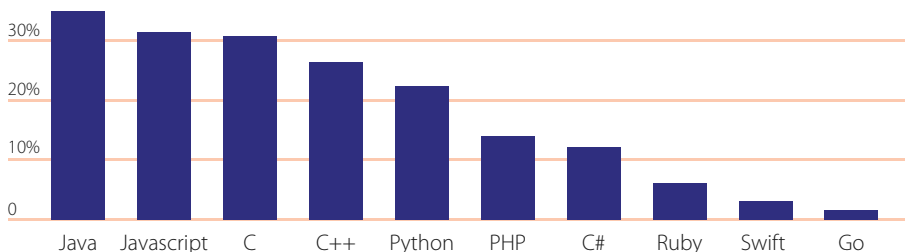
### Theo vị trí công việc



Dữ liệu tham khảo:

- Báo cáo của TopITWorks năm 2017
- Báo cáo của VietnamWorks năm 2018
- Báo cáo của Navigos năm 2017

## Developer nữ đang theo đuổi công nghệ gì?



## Top nguồn học lập trình

77%

HỌC CÁC  
KHÓA HỌC ONLINE

- Stackoverflow.com
- Codecademy.com
- Github.com

33%

## CẬP NHẬT TIN TỨC

- Medium.com
- Techtalk.vn
- Viblo.asia
- Slimshare.com
- Kipalog

29%

THAM GIA CÁC  
SỰ KIỆN CÔNG NGHỆ

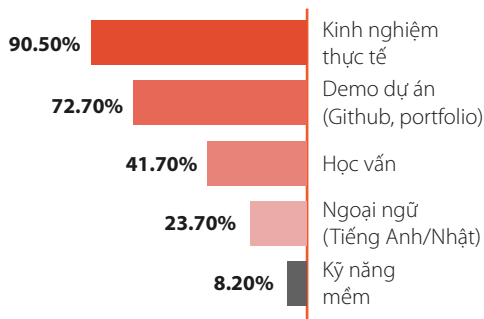
- Vietnam Mobile Day
- Vietnam Web Summit
- TechFest
- Tech Summit

Ngoài những trang nước ngoài, những trang blog tại Việt Nam như Techtalk.vn, Viblo, hay Kipalog chính là nơi mà các lập trình viên có thể trò chuyện và chia sẻ kiến thức của mình. Đặc biệt, Techtalk.vn còn thường xuyên có những sự kiện công nghệ với sự góp mặt chia sẻ của các chuyên gia hàng đầu trong thị trường Tech.

## Nhà tuyển dụng quan tâm điều gì ở CV của Developer

Từng có một quan niệm sai lầm rằng nhà tuyển dụng ưu tiên các ứng viên xuất phát từ các trường CNTT Top đầu thị trường. Nhưng thực tế, điều họ quan tâm chính là kinh nghiệm và năng lực làm việc thực tế của ứng viên.

Ngoại ngữ cũng đang là một kỹ năng bắt buộc của Developer.



## Methodologies thông dụng nhất

85.4%

AGILE

62.7%

SCRUM

35.2%

KANBAN

28.4%

PAIR PROGRAMMING

Agile và Scrum vẫn là những methodologies thông dụng nhất cho developer để quản lý tiến độ của dự án.

## Mức thu nhập của Developer theo số năm kinh nghiệm



\$330 - \$515

**MỚI TỐT NGHIỆP**

(Dưới 2 năm làm việc)



\$525 - \$1150

**CÓ KINH NGHIỆM**

(Trên 2 năm làm việc)



\$1550 - \$2350

**CẤP QUẢN LÝ**

(Dưới 5 năm làm việc)



> \$2300

**GIÁM ĐỐC  
VÀ CẤP CAO HƠN**

(Dưới 10 năm làm việc)

Sau 1 năm làm việc, mức lương của Developer thường sẽ tăng đến mức đáng kể. Các nhà tuyển dụng cũng gặp rất nhiều khó khăn cho việc tuyển dụng những vị trí từ cấp quản lý trở lên do những đòi hỏi và yêu cầu gắt gao.

## Dự đoán nhân lực IT tại Việt Nam 2018 - 2020

2018

**Cần  
320,000  
Nhân lực**

**Thiếu  
75,000  
Nhân lực**

2019

**Cần  
350,000  
Nhân lực**

**Thiếu  
90,000  
Nhân lực**

2020

**Cần  
400,000  
Nhân lực**

**Thiếu  
100,000  
Nhân lực**

Thiếu hụt nhân sự luôn là một bài toán nan giải cho thị trường IT, dù xu hướng lương thưởng và phúc lợi đang tăng mạnh cho ngành này, nhưng nhiều chuyên gia dự báo năm 2019 Việt Nam vẫn sẽ thiếu hụt từ 70.000 đến 90.000 nhân sự IT.

## Mẫu CV cho lập trình viên



**Nguyễn Văn A**  
Full Stack Java Developer

May 05, 1997  
098 333 xxxx  
nguyenvana@gmail.com  
<https://github.com/nguyenvana>  
[linkedin.com/in/nguyenvana](https://www.linkedin.com/in/nguyenvana)

### // OBJECTIVE

I have 3 years experience in Web Development, i'm seeking position of a Junior Java Developer, where i can develop ability myself and develop products for million user.

### // WORK EXPERIENCE

#### CodeGym

Mar 2018 - Present

#### Fresher Java Developer

- Learning Personal Kanban, Scrum Essence
- Learning Java Core, OOP, Data structure, Algorithm
- Learning Servlet, Spring, MySQL
- Making websites using Servlet, Spring, MySQL, MVC, ORM

#### MOG

Aug 2017 - Mar 2018

#### ReactJS Developer

- Making website <https://7saturday.com> using ReactJS Flux, HTML5, SCSS, NodeJS

#### Felizz Pte Ltd

Nov 2016 - Jul 2017

#### Web Developer

- Converting Sketch to HTML, CSS
- Making website <https://seanote.com> using ReactJS, Flux, HTML5, CSS

#### RedSand Company

Sep 2014 - Sep 2016

#### Web Developer

- Converting PSD, AI, FW, Image to HTML/CSS
- Making websites using Wordpress

#### AioTech Company

Dec 2013 - Jun 2014

#### Web Developer

- Making websites using CodeIgniter Framework

### // SKILLS

OOP, Data structure, Algorithm

Java, Spring, MySQL

Scrum, Git

Javascript, React, Flux

HTML5, CSS3

### // EDUCATION

December 2012 - June 2013  
APROTRAIN APTECH EDUCATION

### // CERTIFICATIONS

ACCP i11 - Hà Nội Education  
2015

Personal KanBan - Học Viện Agile  
2018

Scrum Essence - Học Viện Agile  
2018

### // PROJECTS

- **Simple Blog**  
<https://github.com/nguyenvana/spring-blog>  
Using Spring MVC, ORM, Bootstrap
- **Upload, Download File**  
<https://github.com/nguyenvana/spring-download-file>  
Using Spring MVC, ORM, Bootstrap
- **Kanban App**  
<https://github.com/nguyenvana/kanban-app>  
Using ReactJS
- **Life Style**  
<https://github.com/nguyenvana/life-style>  
Using HTML5, CSS3, SCSS, jQuery

## Các câu hỏi thường gặp khi phỏng vấn tại các công ty phần mềm

### Các câu hỏi thông thường

- |                                                                           |                                                                              |                                                                                                      |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <p><b>1</b></p> <p>Hãy giới thiệu về bản thân</p>                         | <p><b>2</b></p> <p>Bạn thích điều gì nhất ở công việc hiện tại?</p>          | <p><b>3</b></p> <p>Thách thức lớn nhất bạn gặp phải trong công việc là gì?</p>                       |
| <p><b>4</b></p> <p>Tại sao bạn muốn từ bỏ công việc hiện tại?</p>         | <p><b>5</b></p> <p>Ông chủ hiện tại của bạn là người như thế nào?</p>        | <p><b>6</b></p> <p>Bạn nghĩ vai trò của (bất kì điều gì trong công việc hiện nay của bạn) là gì?</p> |
| <p><b>7</b></p> <p>Bạn biết gì về công ty của chúng tôi?</p>              | <p><b>8</b></p> <p>Tại sao bạn muốn có công việc này?</p>                    | <p><b>9</b></p> <p>Bạn có khả năng/kinh nghiệm gì để đảm đương công việc?</p>                        |
| <p><b>10</b></p> <p>Bạn hi vọng sẽ làm ở công ty này bao lâu?</p>         | <p><b>11</b></p> <p>Những điểm mạnh nhất của bạn là gì?</p>                  | <p><b>12</b></p> <p>Những điểm yếu nhất của bạn là gì?</p>                                           |
| <p><b>13</b></p> <p>Bạn hình dung mình sẽ ở vị trí nào sau 5 năm nữa?</p> | <p><b>14</b></p> <p>Trong mắt của đồng nghiệp, bạn là người như thế nào?</p> |                                                                                                      |

### Các câu hỏi khó hơn

- |                                                                                          |                                                                  |                                                                                                    |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <p><b>1</b></p> <p>Những sở thích ngoài công việc của bạn là gì?</p>                     | <p><b>2</b></p> <p>Bạn ghét điều gì nhất trong công việc cũ?</p> | <p><b>3</b></p> <p>Những thành tích nổi bật trong công việc hiện tại (gần nhất) của bạn là gì?</p> |
| <p><b>4</b></p> <p>Công việc này phù hợp với kế hoạch sự nghiệp của bạn như thế nào?</p> | <p><b>5</b></p> <p>Mức lương hiện tại của bạn là bao nhiêu?</p>  | <p><b>6</b></p> <p>Bạn hi vọng được trả lương bao nhiêu?</p>                                       |



## Kỹ năng phỏng vấn

“Tim đập chân run” là tình trạng chung của đa số ứng viên khi bắt đầu một cuộc phỏng vấn. Nhưng nếu căng thẳng quá mức, bạn sẽ không thể chứng minh “bản lĩnh” của mình đối với nhà tuyển dụng (NTD). Bạn chỉ cần chú ý một số bí quyết đơn giản sau để không bỏ lỡ cơ hội “lọt vào mắt xanh” của NTD.

### Đừng liệt kê, hãy chứng minh

Ngồi đối diện với bạn là người phỏng vấn. Họ muốn nghe gì từ bạn? Rõ ràng ngoài trách nhiệm công việc, NTD rất muốn nghe những thành tích mà bạn đã đạt được trong công việc trước đây. Nếu NTD hỏi “Anh/chị đã từng quản lý bao nhiêu nhân viên?” đừng đưa ngay con số chính xác. Hãy “đánh bóng” khả năng lãnh đạo của bạn với câu trả lời chi tiết hơn “Ở IBM, tôi quản lý 35 nhân viên. Không chỉ quản lý công việc của nhân viên, tôi còn chịu trách nhiệm tuyển dụng, hướng dẫn và đào tạo nhân viên mới để giúp họ thích nghi với môi trường làm việc. Ngoài ra, tôi còn quyết định mức lương, thưởng cho mỗi nhân viên. Bộ phận chúng tôi đã góp phần tăng doanh số công ty lên 35% chỉ trong vòng một năm”.

### Biến “không thể” thành “có thể”

Bạn sẽ trả lời như thế nào nếu NTD hỏi bạn có biết sử dụng excel thành thạo hay không trong khi bạn thấy thiếu tự tin khi sử dụng phần mềm này? Đừng lắc đầu bảo không ngay lập tức! Hãy nêu những kỹ năng tương tự mà bạn có để “bù đắp” cho khiếm khuyết này. “Tôi có thể sử dụng phần mềm Lotus, vì thế tôi tin mình có thể nhanh chóng sử dụng thành thạo Microsoft excel.” Mặc dù thành thật “thú nhận” nhưng bạn sẽ không mất điểm vì đã chứng minh được với NTD về ý chí sẵn sàng học hỏi của bạn.

## Sử dụng cách trình bày “diễn dịch”

Hãy dùng cách trình bày “diễn dịch” để mô tả thật chính xác và ấn tượng những thành tích của bạn. Nếu nhà tuyển dụng (NTD) yêu cầu bạn chứng minh khả năng quản lý dự án, hãy cho biết bạn có kinh nghiệm này ở những chức vụ nào, nắm giữ những trọng trách gì vv... Sau đó, bạn sẽ đi sâu mô tả cách thực hiện dự án đó, những thành tích mà bạn đã đạt được, nguồn nhân lực bạn đã quản lý, cách phân công công việc và giải quyết những vấn đề nảy sinh trong quá trình thực hiện dự án.

## Luôn ghi nhớ: bạn đã vượt qua vòng phỏng vấn đầu tiên

Bạn đã trải qua vòng phỏng vấn đầu tiên với NTD. Và giờ đây bạn đang ngồi trong phòng phỏng vấn, trước mặt bạn là chủ tịch hội đồng quản trị, giám đốc tài chính. Bạn hồi hộp quá và mồ hôi bắt đầu rịn ra trên trán. Bạn đừng quá căng thẳng. Hãy hít thở thật sâu để lấy lại bình tĩnh và trấn an rằng bạn đã vượt qua được vòng phỏng vấn đầu tiên. Giờ đây, chính những nhân vật quan trọng hàng đầu đang muốn phỏng vấn bạn. Làm thế nào bạn được những nhân vật then chốt này phỏng vấn? Chỉ có một lý do duy nhất: bạn là một ứng viên sáng giá, khiến các sếp dành thời gian để nói chuyện trực tiếp. Vì thế, đây chính là lúc bạn tự tin thể hiện bản lĩnh của mình.

## “Đi trước một bước...”

Buổi phỏng vấn sắp kết thúc, thế nhưng bạn vẫn chưa giới thiệu được kinh nghiệm “ruột” của mình do bạn đã “lỡ” quên trình bày với NTD. NTD cũng có thể không hỏi sâu về một kỹ năng mà bạn rất tâm đắc như “quản lý các kênh phân phối”. Trong trường hợp đó, bạn hãy chủ động “Đề cao” khả năng làm việc của mình. Hãy tranh thủ ngay khi NTD vừa dùng lời để trình bày “Trước khi kết thúc buổi phỏng vấn, tôi muốn nói thêm về kỹ năng quản lý các kênh phân phối sản phẩm. Tôi nghĩ đây là kinh nghiệm giúp quý công ty xác định kỹ năng của tôi xem có phù hợp với vị trí này hay không...” Bạn cần đi trước NTD một bước để chứng tỏ giá trị thật sự của mình.

Hầu hết các cuộc phỏng vấn sẽ làm ứng viên căng thẳng và hồi hộp. Tuy nhiên, đó là “một phần tất yếu” trong quá trình “săn việc”. Nếu biết cách kiểm soát một cuộc phỏng vấn, bạn sẽ nắm trong tay cơ hội tìm được công việc bạn hằng mơ ước.

Theo Navigos





## Nên làm gì để bắt đầu theo đuổi nghề lập trình

Sau khi đã tìm hiểu những điều cơ bản về nghề lập trình, câu hỏi tiếp theo đặt ra là bạn cần bắt đầu từ đâu để trở thành một lập trình viên? Dưới đây là một số lời khuyên CodeGym đưa ra nhằm giúp các bạn sinh viên biết được mong muốn và cách thức các doanh nghiệp đánh giá, tuyển chọn và tuyển dụng nhân viên trong ngành CNTT, đồng thời định hướng giúp các bạn nên bắt đầu như thế nào để có thể chuẩn bị hành trang cho quá trình đi tìm kiếm và lấy được công việc khi ra trường.

### Hiểu được quy trình và tiêu chuẩn đánh giá, lựa chọn ứng viên tuyển dụng của doanh nghiệp để có sự chuẩn bị ngay từ bây giờ

Để đánh giá năng lực của một ứng viên cho vị trí lập trình viên trình độ fresher (mới ra trường), thông thường doanh nghiệp sẽ đánh giá bạn qua 3 nhóm năng lực:

**Kiến thức và kỹ năng lập trình nền tảng:** 12 nhóm kỹ năng dưới đây chính là những kiến thức và kỹ năng lập trình nền tảng mà doanh nghiệp nào cũng yêu cầu ứng viên nắm chắc để qua được vòng sơ loại và bước vào vòng phỏng vấn tiếp theo. Học và nắm chắc nhóm kiến thức và kỹ năng này là một bước thiết yếu bắt buộc đầu tiên để bắt đầu hành trình đến với công việc bạn theo đuổi.

- Cộng tác và làm việc nhóm
- Quy trình sản xuất phần mềm
- Công cụ làm việc
- Lập trình căn bản
- Lập trình Hướng đối tượng
- Thiết kế và Kiến trúc phần mềm
- Cơ sở dữ liệu
- Kiểm thử phần mềm
- Quy ước mã nguồn và mã sạch
- Sử dụng hệ điều hành
- Bảo mật
- Ngôn ngữ lập trình



**Kinh nghiệm và khả năng làm sản phẩm thực tế:** Những gì các bạn đã học thôi chưa đủ để doanh nghiệp đánh giá năng lực làm việc thực tế của bạn. Bạn cần có sản phẩm cụ thể để giới thiệu và chứng minh với doanh nghiệp khả năng vận dụng những kiến thức đã học vào thực tế công việc. Cách chứng minh tốt nhất chính là đưa ra những sản phẩm, những dự án bạn đã làm được. Hãy cố gắng mô tả cho doanh nghiệp thấy được bạn đã làm được gì, vai trò của bạn trong dự án là gì, bạn đã phát triển sản phẩm hay dự án đó như thế nào.

**Thái độ, kỹ năng làm việc, tương tác nhóm:** Trong thời đại ngày nay, các doanh nghiệp luôn nỗ lực tăng hiệu quả công việc thông qua việc tăng hiệu quả phối hợp, làm việc, quản lý công việc của mỗi cá nhân, mỗi nhóm làm việc. Đặc biệt, với các doanh nghiệp sản xuất phẩm mềm thì kỹ năng làm việc của mỗi nhân viên sẽ có ảnh hưởng rất nhiều đến hiệu quả và kết quả công việc chung của cả dự án, cả công ty. Do đó, bạn hãy tìm cho mình một nơi đào tạo kỹ năng làm việc, ứng dụng và thực hành tốt những kỹ năng này. Chắc chắn, điều này sẽ giúp bạn tự tin hơn rất nhiều khi tham gia vào các vòng tuyển dụng cũng như khi chính thức tham gia vào môi trường làm việc.

## **Hãy bắt tay vào ngay từ hôm nay để chuẩn bị cho mình những hành trang nói trên**

**Kiến thức và kỹ năng lập trình nền tảng:** Hãy bám vào 12 nhóm kỹ năng và kiến thức lập trình mà CodeGym đã tư vấn cho bạn trong bài test để xem bạn nên học những gì. Những kiến thức này có thể bạn sẽ phải học ở rất nhiều nơi như trường học của bạn, từ doanh nghiệp, từ các đơn vị đào tạo lập trình. Hãy nhớ rằng bạn phải học được và làm được. Bạn có thể dễ dàng nắm bắt được những kiến thức thông qua khóa học online hay qua một video từ Youtube. Nhưng để làm được, bạn cần phải tìm đến một đơn vị đào tạo, hoặc một người huấn luyện để giúp bạn thực hành được những kiến thức đã học.

**Kinh nghiệm và khả năng làm sản phẩm thực tế:** Có rất nhiều cách để bạn có thể từng bước xây dựng cho mình một hồ sơ kinh nghiệm.

- Bạn có thể nộp hồ sơ xin ứng tuyển vào các chương trình thực tập sinh của doanh nghiệp, ở đó bạn sẽ được học và biết hơn về môi trường làm việc thực tế của doanh nghiệp, và có thể có cơ hội tham gia vào làm dự án thực tế, mặc dù có thể vai trò của bạn rất nhỏ.
- Bạn cũng có thể bắt tay vào làm ngay những sản phẩm cụ thể, nhỏ nhỏ phù hợp với sức mình như tạo ra những ứng dụng đơn giản, tạo một website tĩnh hoặc viết một game nhỏ. Hãy cố gắng hoàn thành sản phẩm dù chỉ ở mức đơn giản nhất, sau đó đưa lên App Store hoặc nếu là website hãy sử dụng một trang cung cấp theme website miễn phí để mọi người có thể nhìn thấy website của bạn. Đây chắc chắn sẽ là những "bằng chứng" rất sinh động thể hiện sự năng động và nghiêm túc của các bạn. Hãy nhớ rằng khi tuyển dụng fresher, doanh nghiệp không đòi hỏi bạn phải có kinh nghiệm làm việc lâu hay phải có sản phẩm lớn. Điều họ muốn đánh giá chính là "khả năng" làm việc thực tế của các bạn. Nếu bạn tự chủ động và đủ khả năng làm ra sản phẩm cho chính mình thì chắc chắn bạn sẽ là một ứng viên "có khả năng" làm việc được. Nếu bạn không biết bắt đầu từ đâu, hãy tìm ngay cho mình một người đỡ đầu, một người huấn luyện, hoặc một nơi đào tạo thực tế có thể giúp các bạn từng bước luyện tập, làm được sản phẩm và tự tin tham gia vào các nhóm dự án phát triển sản phẩm trên thực tế tại các doanh nghiệp.
- Ngoài ra, nếu bạn có một nhóm bạn hoặc có dự án nào mà bạn có khả năng tham gia, hãy xin tham gia, kể cả không được trả công và bất kể vai trò như thế nào. Tham gia tích cực, nghiêm túc và ghi chép cẩn thận quá trình thực hiện dự án, mô tả phần công việc của bạn và những kiến thức, kinh nghiệm rút ra được từ dự án. Chắc chắn đây sẽ là kinh nghiệm mà nhà tuyển dụng mong muốn được lắng nghe nhất để đánh giá ứng viên tiềm năng.

**Dù bạn định hướng đi theo chuyên môn BA, AI hoặc IoT hay là lập trình viên, hay thậm chí bạn chưa biết mình thích làm gì, đừng suy nghĩ gì nhiều, hãy bắt đầu ngay từ hôm nay để chuẩn bị thật tốt những hành trang chúng tôi đã tư vấn ở trên. Sau đó, bắt đầu ngay với một công việc mà bạn nhận được. Làm việc thật nghiêm túc, không ngừng học hỏi tiếp và bạn sẽ nhận ra con đường đi của mình**

Hy vọng rằng những tư vấn trên có thể bước đầu giúp bạn định hướng trên con đường theo đuổi nghề nghiệp của một lập trình viên chuyên nghiệp. Nếu bạn cần tư vấn, cần một nơi để có thể được hỏi, được học hỏi, hay đơn giản chỉ để được chia sẻ niềm đam mê công nghệ và lập trình, bạn có thể đến với CodeGym. Luôn luôn có rất nhiều chuyên gia, những người bạn sẵn sàng hỗ trợ bạn nhé.

## Hãy đọc sách

Hãy cố gắng đọc thật nhiều sách nhất có thể để tìm hiểu và trau dồi thêm kiến thức về chuyên môn lập trình, các kỹ năng nghề nghiệp cần thiết cho công việc lập trình viên của bạn. Dưới đây là một số quyển sách hay các bạn có thể đọc:

### Sách phát triển tư duy nghề nghiệp

- Apprenticeship Patterns
- Soft Skills:  
The Software Developer's Life Manual
- Coders at Work:  
Reflections of the Craft of Programming
- Code đạo кі sự
- AgileY
- The Clean Coder: A Code of Conduct for Professional Programmers
- The Pragmatic Programmer:  
From Journeyman to master
- Pragmatic Thinking and learning
- Clean Code: A Handbook of Agile Software Craftsmanship
- Code Complete: A Practical Handbook of Software Construction

### Sách phát triển cá nhân

- Đọc sách như một nghệ thuật (Phương pháp đọc sách hiệu quả)
- Những kẻ xuất chúng
- Mật mã tài năng
- Hoàn thành mọi việc không hề khó
- Sức mạnh của thói quen
- Tư duy như hệ thống (the organized mind)
- Nghệ thuật tư duy rành mạch
- Tâm lý học thành công (Mind set)  
Cách sống  
Ba người thầy vĩ đại

## Về CodeGym

CodeGym là hệ thống đào tạo lập trình hiện đại dành cho mọi đối tượng từ những người mới bắt đầu, sinh viên công nghệ thông tin đến các lập trình viên có tay nghề.

### Chương trình đào tạo hiện đại chuẩn quốc tế

CodeGym là hệ thống đào tạo duy nhất tại Việt Nam xây dựng riêng chương trình đào tạo phát triển phần mềm hiện đại theo tiêu chuẩn quốc tế đang được các doanh nghiệp hàng đầu thế giới như Google, Facebook áp dụng: Agile, Scrum, DevOps.

### Nền tảng công nghệ đào tạo và học tập ưu việt

Hệ thống kho tri thức CodeGym X, phòng thực hành CodeGym Lab, công cụ luyện code CodeGym Bob cùng sự huấn luyện và dẫn dắt 100% thời gian của các huấn luyện viên là các chuyên gia đầu ngành đến từ các doanh nghiệp, các trường đào tạo CNTT hàng đầu tại Việt Nam và khu vực.

### Doanh nghiệp đồng hành đào tạo và tuyển dụng học viên

Doanh nghiệp hợp tác với CodeGym tham gia vào quá trình đào tạo từ khi thiết kế chương trình theo nhu cầu và xu thế công nghệ của thị trường, đến đánh giá chất lượng học viên qua các giai đoạn và tuyển dụng sau khi học xong.





## Trở thành lập trình viên chuyên nghiệp sau 4 tháng với Coding Bootcamp

### Bootcamp

Chương trình CodeGym Coding Bootcamp được xây dựng nhằm đào tạo những lập trình viên hiện đại. Hoàn thành khoá học, học viên có đủ năng lực của một lập trình viên chuyên nghiệp, có thể tham gia trực tiếp vào các dự án phần mềm tại doanh nghiệp hoặc tự mình xây dựng các ứng dụng, phần mềm khác nhau.

Để đào tạo thành công các học viên ít hoặc không có nền tảng công nghệ thành những lập trình viên full stack chuyên nghiệp và nhận được việc làm ngay khi học xong chỉ sau 16 tuần, CodeGym đầu tư bài bản cho hoạt động nghiên cứu, phát triển và xây dựng chương trình học riêng phù hợp với đặc điểm người học theo các trình độ khác nhau và yêu cầu thực tế của doanh nghiệp Việt Nam.

Đồng thời, CodeGym hợp tác chặt chẽ với doanh nghiệp để đánh giá và nâng cao chất lượng đào tạo, đảm bảo việc làm.

Thông qua công nghệ giáo dục, ứng dụng công nghệ thông tin, lộ trình học tập và huấn luyện được cá nhân hoá đến từng học viên thông qua hệ thống phần mềm học tập hiện đại, các huấn luyện viên và giảng viên kèm 1-1 trong 100% thời gian đào tạo nhằm hỗ trợ và đảm bảo học viên hoàn thành được mục tiêu học tập.

# Lộ trình học tập





## Coding Bootcamp đầu tiên tại Việt Nam

Đào tạo lập trình viên chuyên nghiệp theo phương pháp huấn luyện thực chiến:

- Nội dung sát với yêu cầu thực tế của doanh nghiệp
- Thời gian đào tạo ngắn
- Cường độ huấn luyện cao
- Thực hành liên tục
- Huấn luyện cá nhân hoá, hỗ trợ tối đa cho tới khi làm được việc

## Cam kết việc làm

Bạn sẽ được đào tạo toàn diện các kỹ năng nghề nghiệp và sẵn sàng làm việc ngay khi học xong. CodeGym cam kết học viên sẽ nhận được việc làm trong 45 ngày sau khi kết thúc khoá học.

Các vị trí lập trình bạn có thể đảm nhiệm:

- LTV Back-end
- LTV Front-end
- LTV Full-stack



## Cú pháp cơ bản

### Nhúng PHP

```
<p>HTML 1</p>
<?php echo 'HTML-output from PHP.'; ?>
<p>HTML 2</p>
```

### Các biến

```
$name = "Anthony";
$pointerName = &$name;
$a = "bc";
$a = "a"; //Biến tên bc có giá trị là "a"
Sử dụng với nhiều mảng: ${$a[1]} hoặc ${a}[1]
Hằng số: define("constantA", "value");
```

### Các biến được xác định

**\$GLOBALS:** Biến toàn cục.

**\$\_SERVER:** Lưu trữ thông tin về máy chủ và môi trường thực thi.

**\$\_GET:** Lưu trữ các dữ liệu của phương thức GET Client gửi lên.

**\$\_POST:** Lưu trữ các dữ liệu của phương thức POST Client gửi lên.

**\$\_FILES:** Lưu trữ thông tin của tệp được tải lên.

**\$\_REQUEST:** Lưu trữ các dữ liệu được gửi lên bao gồm cả phương thức GET và POST.

**\$argc:** Lưu trữ số lượng đối số được truyền lên.

**\$argv:** Lưu trữ mảng các đối số được truyền lên.

**\$\_SESSION:** Chứa Session.

**\$\_ENV:** Biến môi trường.

**\$\_COOKIE:** Lưu trữ dữ liệu cookie nhận được từ trình duyệt người dùng.

**\$php\_errormsg:** Biến lưu trữ thông báo lỗi cuối cùng.

**\$HTTP\_RAW\_POST\_DATA:** Lưu trữ dữ liệu thô của phương thức POST.

**\$HTTP\_response\_header:** Lưu trữ 1 mảng các tiêu đề phản hồi từ HTTP.

### Các hàm

Kiểm tra kiểu của dữ liệu: `is_array`, `is_binary`, `is_bool`, `is_buffer`, `is_callable`, `is_double`, `is_int`, `is_float`, `is_integer`, `is_long`, `is_null`, `is_numeric`, `is_object`, `is_real`, `is_resource`, `is_string`, `is_scalar`, `is_unicode`, `gettype`, `get_resource_type`.

Kiểm tra: `empty`, `isset`.

Cast: `floatval`, `intval`, `strval`, `settype`.

Đầu ra: `var_dump`, `var_export`, `print_r`.

Xóa: `unset`.

Tuần tự hóa: `serialize`, `unserialize`.

### Các lệnh bổ sung

| Lệnh | Ý nghĩa         | Mã ASCII        |
|------|-----------------|-----------------|
| \n   | Xuống dòng      | LF or 0x0A (10) |
| \r   | Về đầu dòng     | CR or 0x0D (13) |
| \t   | Ký tự tab ngang | HT or 0x09 (9)  |
| \v   | Ký tự tab dọc   | VT or 0x0B (11) |
| \f   | Form feed       | FF or 0x0C (12) |
| \\   | Ký tự \         |                 |
| \\$  | Ký tự \$        |                 |
| \"   | Ký tự "         |                 |



Mảng

Khai báo

Sử dụng array() để tạo mảng  
Xây dựng mảng  
array([ mixed \$...]),  
array( key => value, ...)  
Hoặc \$array[key] = value;  
\$arr[] = value;

Kiểu của mảng

| Mảng số                                                                                                                                                                                                | Assoziatives Array                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Sử dụng của mảng \$farben = array("red", "green"); // Gán giá trị // Chỉ số bắt đầu từ 0 \$farben[] = "red"; \$farben[] = "green"; // Truy cập Echo \$color[1]; // Kết quả in ra "green"</pre> | <pre>// Sử dụng của mảng \$sale = array("Jan" =&gt; 100              "Feb" =&gt; 120); // Gán giá trị \$sales["Jan"] = 100; \$sales["Feb"] = 120; // Truy cập Echo \$umsaetze["Jan"];</pre> |

Các hàm

array\_flip: Đảo ngược lại key và value.  
array\_intersect: Tạo giao điểm các phần tử.  
array\_pop: Truy xuất phần tử cuối cùng.  
array\_push: Thêm phần tử vào cuối mảng.  
array\_rand: Lấy các phần tử ngẫu nhiên.  
array\_reverse: Đảo ngược thứ tự các phần tử.  
array\_search: Tìm kiếm một mảng và truy xuất các khóa.  
array\_shift: Truy xuất phần tử đầu tiên.  
array\_slice: Trích xuất một tập hợp con.  
count: Đếm các phần tử.  
in\_array: Kiểm tra sự xuất hiện của một giá trị.  
shuffle: Phân tách các phần tử.



## Cấu trúc điều khiển

### Câu lệnh điều khiển

Các câu lệnh điều khiển kiểm tra một biểu thức boolean và câu lệnh thực thi tùy thuộc vào đầu ra kiểm tra là đúng hay sai. Các biểu thức có thể kết hợp nhiều biểu thức với dấu && (và) hoặc || (hoặc).

Ví dụ:

```
$expr = true;
if (expr) echo "singleStatementInNextRow";
$number1 = 5;
$number2 = 2;
if ($number1 = 4){
    // statements
}

elseif ($number1 >= 4 && $ number2 < 3){
    // statements
} else {
    // statements
}

switch ($animal) {
    case "Dog":
        // statements break;
    case "Cat":
        // statements break;
    default:
        // mặc định sẽ thực thi khi không có case nào đúng
}
```

### Câu Lệnh Điều Khiển

Lệnh break dừng việc thực thi của vòng lặp. Lệnh continue bỏ qua việc thực hiện các câu lệnh và tiếp tục vòng lặp.

Ví dụ:

```
for ($i = 0; $i < 10; $i++){
    if ($i == 2) continue;
    if ($i % 2 == 0 || $i == 5) {
        echo $i."<br/>";
    }
    if ($i > 8) break;
}
```

### Bao Gồm Tập Lệnh

Các câu lệnh require("file.php") và include ("file.php") có chức năng chèn thêm các tệp PHP. Khi có lỗi nếu sử dụng require thì sẽ tạo một cảnh báo và nếu sử dụng include thì sẽ tạo một lỗi nghiêm trọng. Các câu lệnh require\_once và include\_once chỉ thực hiện đưa vào cùng một tệp một lần.

## Vòng lặp

Các vòng lặp thực hiện các câu lệnh liên tục cho đến khi điều kiện là đúng.

Ví dụ:

```
$number = 1;
while ($number <= 5){
    echo $number."<br/>";
    $number++;
}
for ($i = 0; $i < 10; $i++){
    echo $i."<br/>";
}
```

Thực hiện các câu lệnh một lần và kiểm tra điều kiện ngay sau đó. Nếu điều kiện là đúng thì sẽ lặp lại thực thi câu lệnh cho đến khi điều kiện sai.

Ví dụ:

```
$number = 10;
do {
    echo $number."<br/>";
    $number ++;
}
while ($number <=5);
```

Duyệt qua các phần tử của mảng và thực hiện các câu lệnh cho mỗi lần lặp. Chỉ số của mảng sẽ được gán tạm thời cho biến.

Ví dụ:

```
$celBodies = array("sun"
=> "yellow", "moon"
=> "pale", "stars" => "white");
foreach ($celBodies as $name => $celBody){
    echo $name."<br/>";
}
```

## Ngoại lệ

Một ngoại lệ là một thông báo lỗi. Lớp cơ sở cho các trường hợp ngoại lệ đặc biệt thường là exception. Một ngoại lệ được ném (kích hoạt) bởi throw. Một khối trong ngoại lệ là throw được xử lý bên trong khối try{} và ngoại lệ được xử lý trong khối catch{}.

Ví dụ:

```
// Throwing an exception
throw new Exception
("Msg text",123);
// Method call and exception
handling
try{
    // Statements which may
    cause exception
}catch (MyException $e){
    echo $e->getMessage().
    "(".$e->getCode().")";
}catch (Exception $e){
    echo $e->getMessage().
    "(".$e->getCode().")";
}
// Declaring a specialized
exception class MyException
extends Exception{
    public function __construct
    ($message = "Msg text",
    $code = 123){
        $this->message = $message;
        $this->code = $code;
    }
    // Additional methods ... }
```



## Java

### Chú thích

// Mọi nội dung trong dòng này sẽ bị bỏ qua. Đây là loại chú thích được sử dụng phổ biến.  
 /\* Mọi thứ (có thể gồm nhiều dòng) trong khối này sẽ bị bỏ qua \*/  
 Không phổ biến. Sử dụng để chú thích bên ngoài mã nguồn.  
 /\*\* Được sử dụng để giúp tự động tạo ra Javadoc dưới dạng HTML \*\*/

### Định danh/Đặt tên

- Các định danh phải bắt đầu với ký tự bảng chữ cái Alphabetic (a-z hoặc A-Z), và các ký tự tiếp theo là chữ cái, chữ số (0-9), hoặc dấu gạch dưới (\_). Không sử dụng ký tự '\$'.
- Các từ tiếp theo trong định danh (nếu có) nên bắt đầu với ký tự chữ cái in hoa.
- Không sử dụng các từ khóa của Java.
- Tên lớp và giao tiếp (interface) nên bắt đầu với ký tự chữ cái in hoa (Graphics, String, Car, Motorbike, ...).
- Tên biến và phương thức nên bắt đầu với ký tự chữ cái in thường (repaint(), x, ...).
- Các hằng giá trị nên sử dụng toàn bộ chữ in hoa và dùng dấu gạch dưới (\_) để phân cách các từ (BoxLayout.X\_AXIS, Math.PI, ...).

### Các kiểu dữ liệu nguyên thủy

boolean (với chỉ hai giá trị true/false).  
 Các kiểu số học: byte, short, char, int, long, float, double.

### Biểu thức

Cặp ngoặc tròn () có 3 tác dụng:

- Nhóm để điều khiển thứ tự tính toán, hoặc để biểu thức rõ ràng. VD:  $(a+b) * (c-d)$
- Đặt sau tên phương thức để chứa các tham số. VD: `x = sum(a, b);`
- Chứa tên kiểu dữ liệu khi tiến hành ép kiểu. VD: `i = (int)x;`

### Độ ưu tiên của các toán tử

|                                                                                                        |                                         |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------|
| 1. Toán tử có độ ưu tiên cao hơn sẽ được thực hiện trước.                                              | Chỉ cần nhớ thứ tự sau                  |
| 2. Thứ tự sẽ từ trái qua phải nếu các toán tử có cùng độ ưu tiên, ngoại trừ: một ngôi, gán, điều kiện. | 1. các toán tử một ngôi                 |
|                                                                                                        | 2. * / %                                |
|                                                                                                        | 3. + -                                  |
|                                                                                                        | 4. các toán tử so sánh                  |
|                                                                                                        | 5. &&                                   |
|                                                                                                        | 6. = các toán tử gán                    |
|                                                                                                        | Sử dụng cặp () cho tất cả những toán tử |

### So sánh các giá trị nguyên thủy

Kết quả của các toán tử so sánh là boolean (true hoặc false).  
 ==, !=, <, <=, >, >=

Biến cục bộ, biến đối tượng, biến tĩnh

Biến có thể là biến cục bộ (local), biến đối tượng (instance), hoặc biến tĩnh (static hay còn gọi là biến lớp). Các tham số được coi là những biến cục bộ, chúng được gán giá trị khi phương thức được triệu gọi.

|                          | Biến cục bộ                                                                                   | Biến đối tượng                                                                                                                                                                                                        | Biến tĩnh                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Nơi khai báo?            | Trong phương thức.                                                                            | Trong lớp, nhưng không trong phương thức.                                                                                                                                                                             | Trong lớp và sử dụng thêm từ khóa static.                                                       |
| Giá trị khởi tạo         | Phải được gán giá trị trước khi sử dụng và trình biên dịch sẽ báo lỗi nếu không làm việc này. | Kiểu số: 0<br>Đối tượng: null<br>Boolean: false<br>Hoặc được khởi tạo giá trị trong hàm tạo.                                                                                                                          | Kiểu số: 0<br>Đối tượng: null<br>Boolean: false<br>Hoặc được khởi tạo trong khối khởi tạo tĩnh. |
| Phạm vi sử dụng          | Chỉ trong phương thức chứa nó. Không có khả năng hiển thị có thể được khai báo.               | private: Chỉ các phương thức của lớp.<br>(Mặc định): Tất cả các phương thức thuộc các lớp cùng gói (package).<br>public: Bất cứ đâu đều có thể truy xuất.<br>protected: Lớp chứa nó và tất cả các lớp con (subclass). | Giống với Biến đối tượng.                                                                       |
| Được tạo ra khi nào?     | Khi phương thức được sử dụng.                                                                 | Khi một đối tượng của lớp được tạo mới.                                                                                                                                                                               | Khi chương trình được nạp.                                                                      |
| Được lưu trữ ở đâu?      | Stack                                                                                         | Heap                                                                                                                                                                                                                  | Vùng nhớ "Bền vững".                                                                            |
| Khi nào được giải phóng? | Khi phương thức trả về kết quả.                                                               | Khi không còn tham chiếu nào dành cho đối tượng.                                                                                                                                                                      | Khi chương trình kết thúc.                                                                      |

Các toán tử số học

Kết quả của các toán tử số học là double nếu có một toán hạng là double, trái lại là float nếu có một toán hạng là float, trái lại là long nếu có một toán hạng là long, trái lại là int.

i++ Cộng thêm 1 cho i

i-- Bớt 1 từ i

n+m Phép cộng. VD: 7 + 5 bằng 12, 3 + 0.14 bằng 3.14

n-m Phép trừ.

n\*m Phép nhân. VD: 3 \* 6 bằng 18

n/m Phép chia. VD: 3.0 / 2 bằng 1.5, 3 / 2 bằng 1

n%m Phép chia lấy phần dư (Mod). VD: 7 % 3 bằng 1

## Các toán tử lô-gíc

Các toán hạng phải có kiểu là boolean. Kết quả là boolean.

`b & c` “Và”. Kết quả là true nếu cả hai toán hạng đều là true, trái lại sẽ là false.

Đánh giá đoản mạch. VD: *(false && anything)* cho kết quả false.

`b | c` “Hoặc”. Kết quả là true nếu bất cứ toán hạng nào true, trái lại sẽ là false.

Đánh giá đoản mạch. VD: *(true || anything)* cho kết quả là true.

`!b` “Phủ định”. Kết quả là true nếu b là false, là false nếu b là true.

## Các toán tử gán

= Về trái bắt buộc phải là một định danh/biến.

`+= -= *= ...`

Tất cả các toán tử hai ngôi (ngoại trừ `&&` và `||`) đều có thể kết hợp với toán tử gán.

VD: *a += 1 tương tự như a = a + 1*

## Ép kiểu

Ép kiểu được sử dụng khi “thu hẹp” dải giá trị nào đó. Phạm vi từ hẹp nhất đến rộng nhất của kiểu dữ liệu nguyên thủy là: byte, short, char, int, long, float, double. Các đối tượng có thể được gán mà không cần ép kiểu lên cấp cao hơn trong cây phân cấp kế thừa. Ép kiểu chỉ cần khi ép xuống cấp thấp hơn trong cây phân cấp (ép xuống).

(t)x Ép kiểu của x sang kiểu t.

## Toán tử đối tượng

`co.f` Thành phần. Thuộc tính hoặc phương thức f của đối tượng hoặc lớp co.

`x instanceof co` Cho kết quả true nếu đối tượng tham chiếu bởi x là thể hiện của lớp co.

`s+t` Toán tử cộng chuỗi nếu một hoặc cả hai toán tử có kiểu là String.

`x==y` Cho kết quả là true nếu cả x và y cùng tham chiếu tới một đối tượng, trái lại sẽ là false (thậm chí cả khi giá trị của các đối tượng này giống nhau!).

`x!=y` Phủ định của toán tử trên.

**Lưu ý:** So sánh các đối tượng sử dụng phương thức `.equals()` hoặc `.compareTo()`.

`x=y` Sao chép tham chiếu đối tượng chứ không phải sao chép đối tượng.



## Các cấu trúc điều khiển

### Lệnh if

```
// lệnh if với mệnh đề true
if(biểu_thức){
    Các lệnh // thực hiện nếu biểu thức cho kết quả true
}

// lệnh if với mệnh đề true và false
if(biểu_thức){
    Các lệnh // thực hiện nếu biểu thức cho kết quả true
}else{
    Các lệnh // thực hiện nếu biểu thức cho kết quả false
}

// Lệnh if với nhiều điều kiện kiểm tra song song
if(biểu_thức_1){
    Các lệnh // thực hiện nếu biểu thức 1 là true
}else if(biểu_thức_2){
    Các lệnh // thực hiện nếu biểu thức 2 là true
}else if(biểu_thức_3){
    Các lệnh // thực hiện nếu biểu thức 3 là true
}else{
    Các lệnh // thực hiện khi các biểu thức trên là false
}
```

### Lệnh lặp while

```
while(biểu_thức){
    // thực hiện lặp lại các mệnh lệnh cho tới khi
    // biểu thức cho kết quả false
}
```

### Lệnh lặp for

```
for(Khởi tạo biến đếm; điều kiện lặp; thay đổi biến đếm) {
    // thực hiện lặp lại các mệnh lệnh cho tới khi
    // biểu thức điều kiện lặp cho kết quả false
}
```

While và for có thể thay thế cho nhau:

```
int i = 0;
while(i<5){
    System.out.print("Hi!");
    i++;
}
```

```
for (i=0; i<5; i++) {
    System.out.print("Hi!");
}
```

## Lệnh switch

switch cho phép chọn một trường hợp nào đó phụ thuộc vào giá trị nhận được (thường là số nguyên) từ biến hoặc biểu thức.

```
switch (biểu_thức) {
    case c1:
        Các lệnh // thực hiện khi biểu thức = c1
        break;
    case c2:
        Các lệnh // thực hiện khi biểu thức = c2
        break;
    case c3:
    case c4:
    case c5: // các trường hợp cùng chung xử lý.
        // thực hiện khi biểu thức = c3, c4 hoặc c5
        Các lệnh
        break;
    default:
        Các lệnh
        // thực hiện khi mà biểu thức không bằng giá trị nào ở trên
}
```

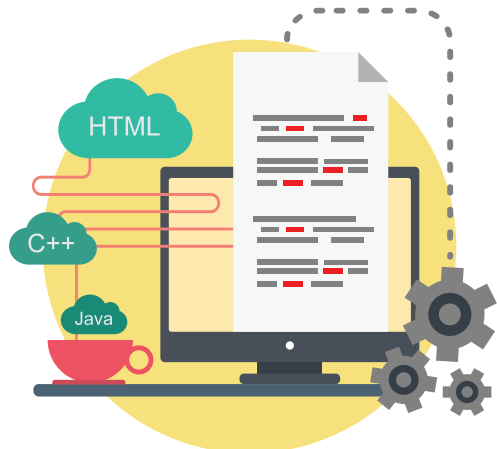
## Những điều khiển lặp khác

Tất cả các lệnh lặp đều có thể được đặt nhãn, vì vậy có thể dùng lệnh break và continue cho bất cứ cấp độ lồng nhau nào của vòng lặp.

```
break; //thoát khỏi vòng lặp hoặc lệnh switch gần nhất chứa nó
break label; // thoát khỏi vòng lặp được đặt nhãn (label)
continue; //tiến hành lượt lặp tiếp theo
continue label; //tiến hành lượt lặp tiếp theo với vòng lặp được
đặt nhãn (label).
```

Nhãn được đặt trước vòng lặp cùng dấu hai chấm, ví dụ:

```
outer:
for (...) {
    ...
    continue outer;
}
```





## Ngoại Lệ

Sử dụng try...catch cho các ngoại lệ

```
try{
    ...// các lệnh có thể dẫn tới ngoại lệ
}catch(Kiểu_ngoại_lệ x){
    ...// các lệnh để xử lý khi có ngoại lệ
}
```

## Throw

```
throw đối_tượng_ngoại_lệ;
```

## Sử dụng nhiều mệnh đề catch và mệnh đề finally

Thực hiện mệnh đề catch đầu tiên tương ứng với ngoại lệ được chỉ ra bởi lớp ngoại lệ hoặc các lớp ngoại lệ cha. Mệnh đề finally luôn được thực hiện (bất kể là có hay không có ngoại lệ xảy ra) vì vậy các tài nguyên sử dụng trước đó có thể được giải phóng (ví dụ, đóng tệp tin):

```
try{
    ... // các lệnh có khả năng xảy ra ngoại lệ
}catch(kiểu_ngoại_lệ x){
    ... // cách lệnh xử lý ngoại lệ
} catch (kiểu_ngoại_lệ x) {
    ...// cách lệnh xử lý ngoại lệ
}finally{
    // các lệnh luôn luôn được thực hiện
    // bất kể có xảy ra ngoại lệ hay không
    ...
}
```

## Chuỗi

### Cộng chuỗi

Toán tử + cho phép nối hai chuỗi lại với nhau. Nếu một toán hạng là String, toán hạng còn lại sẽ được chuyển đổi sang kiểu String và sau đó được cộng với nhau. Đây thường là cách dùng để chuyển đổi các số sang String.

Nếu một đối tượng không phải là String khi được cộng với String phương thức toString() của nó sẽ được triệu gọi. Một tiện ích dành cho việc ra soát bug đó là viết trong các lớp của bạn phương thức toString().

i=s.length() cho phép để xác định độ dài của chuỗi s.

|                  |           |
|------------------|-----------|
| "abc"+"def"      | "abcdef"  |
| "abc"+4          | "abc4"    |
| "1"+2            | "12"      |
| "xyz" + (2+2==4) | "xyztrue" |
| 1+"2.5"          | "12.5"    |

## So sánh chuỗi (sử dụng thay thế cho toán tử == và !=)

`i=s.compareTo(t)` được sử dụng để so sánh chuỗi `s` với `t`.

trả về <0 nếu `s < t`, 0 nếu `s == t`, >0 nếu `s > t`

`i=s.compareToIgnoreCase(t)` tương tự như trên nhưng không phân biệt ký tự in hoa, in thường.

`b=s.equals(t)` cho kết quả true nếu hai chuỗi có cùng giá trị.

`b=s.equalsIgnoreCase(t)` tương tự như trên nhưng không phân biệt ký tự in hoa, in thường.

`b=s.startsWith(t)` cho kết quả true nếu chuỗi `s` chứa chuỗi `t` ở đầu.

`b=s.endsWith(t)` cho kết quả true nếu chuỗi `s` chứa chuỗi `t` ở cuối.

## Tim kiếm trong chuỗi (tất cả các phương thức "indexOf" đều trả về -1 nếu không tìm thấy kết quả)

`i=s.indexOf(t)` trả về vị trí đầu tiên xuất hiện chuỗi `t` trong chuỗi `s`.

`i=s.indexOf(t, i)` trả về vị trí đầu tiên tính từ `i` xuất hiện chuỗi `t` trong chuỗi `s`.

`i=s.lastIndexOf(t)` trả về vị trí cuối cùng xuất hiện chuỗi `t` trong chuỗi `s`.

`i=s.lastIndexOf(t, i)` trả về vị trí cuối cùng tính từ `i` xuất hiện chuỗi `t` trong chuỗi `s`.

`b=s.endsWith(t)` cho kết quả true nếu chuỗi `s` chứa chuỗi `t` ở cuối.

## Lấy một phần của chuỗi

`c=s.charAt(i)` trả về ký tự tại vị trí `i` trong chuỗi `s`.

`sl=s.substring(i)` trả về một chuỗi con của chuỗi `s` tính từ vị trí `i` đến cuối.

`sl=s.substring(i, j)` trả về một chuỗi con của chuỗi `s` tính từ vị trí `i` đến ngay trước vị trí `j`.

## Tạo mới một chuỗi từ chuỗi ban đầu

`sl=s.toLowerCase()` tạo một chuỗi mới ở dạng in hoa toàn bộ.

`sl=s.toUpperCase()` tạo một chuỗi mới ở dạng in thường toàn bộ.

`sl=s.trim()` cắt bỏ các ký tự trắng ở đầu và cuối chuỗi.

`sl=s.replace(cs2, cs3)` thay thế tất cả các chuỗi con `cs2` trong `s` bằng chuỗi `cs3`.

## StringBuilder

Khả năng sửa đổi nhanh hơn so với `String`, sử dụng bộ nhớ và CPU hiệu quả hơn.

`sb=new StringBuilder()` Tạo một `StringBuilder` rỗng.

`sb=new StringBuilder(s)` Tạo một `StringBuilder` từ chuỗi `s`.

`sb=sb.append(x)` Thêm `x` (bất kể kiểu dữ liệu) vào cuối `sb`.

`sb=sb.insert(offset, x)` Chèn `x` (bất kể kiểu dữ liệu) vào `sb` tại vị trí `offset`.

`sb=sb.setCharAt(index, c)` Thay thế ký tự tại vị trí `index` bằng ký tự `c`.

`sb=sb.deleteCharAt(i)` Xóa ký tự tại vị trí `i`.

`sb=sb.delete(beg, end)` Xóa các ký tự từ vị trí `beg` tới vị trí `end`.

`sb=sb.reverse()` Đảo ngược nội dung.

`sb=sb.replace(beg, end, s)` Thay thế các ký tự từ vị trí `beg` tới vị trí `end` bằng `s`.

Có các phương thức `indexOf`, `lastIndexOf`, `charAt`, `equals`, `substring` giống như `String`!

## Mảng

Sử dụng để thao tác với nhiều phần tử dữ liệu, có thể là dữ liệu nguyên thủy hoặc các đối tượng. Tất cả các phần tử phải cùng kiểu dữ liệu. Mảng không có khả năng mở rộng!

Ví dụ:

```
int [] scores;//Khai báo scores là mảng các số nguyên.
scores = new int[12];//Khởi tạo vùng nhớ cho mảng với 12 phần tử.
int[] scores = new int[12];//Kết hợp cả khai báo và khởi tạo.
```

## Khởi tạo một mảng

Nếu các phần tử của mảng không được gán giá trị khởi tạo, chúng sẽ được khởi tạo là 0 với các mảng số, null với các mảng tham chiếu đối tượng, và false với các mảng boolean.

Tạo mảng vào khởi tạo giá trị cho phần tử mảng với một dòng lệnh:

```
String[] names={"Mickey", "Minnie", "Donald"};
```

Hoặc có thể tách thành nhiều dòng lệnh:

```
String[] names=new String[3];
names[0]="Mickey";
names[1]="Minnie";
names[2]="Donald";
```

## Truy cập phần tử mảng

```
scores[5]=86;//Gán cho phần tử có chỉ số là 5 giá trị 86.
scores[i]++;//Tăng một đơn vị cho phần tử có chỉ số là i.
```

## Duyệt lần lượt qua các phần tử mảng

Kích thước của một mảng có thể được xác định bằng cách sử dụng thuộc tính length của chúng.

VD: scores.length i.

## Mảng

### Duyệt lần lượt các phần tử mảng

```
//Sử dụng vòng lặp for chuẩn.
int[] scores=new int[12];
...Khởi tạo giá trị cho mảng
scores
int total=0;
for(int i=0;i<scores.length;i++)
{
total += scores[i];
}
```

```
//Sử dụng vòng lặp for nâng cao.
int[] scores=new int[12];
...Khởi tạo giá trị cho mảng
scores
int total=0;
for(int scr:scores){
total+=scr;
}
```

## Mảng hai chiều

Hầu hết được xử lý với vòng lặp for lồng nhau. Ví dụ:

```
static final int ROWS=2; static
final int COLS=4;
...
int[][] a2=new int[ROWS][COLS];
//...Hiển thị các phần tử mảng trong khung hình chữ nhật
for(int i=0; i<ROWS; i++){
    for(int j=0; j<COLS; j++){
        System.out.print(" "+a2[i][j]);
    }
    System.out.println("");
}
```

## Scanner

Tác dụng chính của java.util.Scanner là giúp đọc các giá trị từ System.in hoặc từ tệp tin.

`sc=new Scanner(System.in);` Sử dụng để đọc dữ liệu từ System.in

`sc=new Scanner(s);` Sử dụng để đọc giá trị từ chuỗi s.

## Các phương thức lấy dữ liệu phổ biến nhất

`s=sc.next()` Trả về "token" tiếp theo, nhiều hơn hoặc ít là một "word".

`s=sc.nextLine()` Trả về toàn bộ một dòng dữ liệu dưới dạng một chuỗi.

`x=sc.nextXYZ()` Trả về giá trị có kiểu là XYZ:

Int, Double, Boolean, Byte, Float, Short

`b=sc.hasNext()` Trả về true nếu còn "token" để đọc.

`b=sc.hasNextLine()` Trả về true nếu còn dòng dữ liệu để đọc.

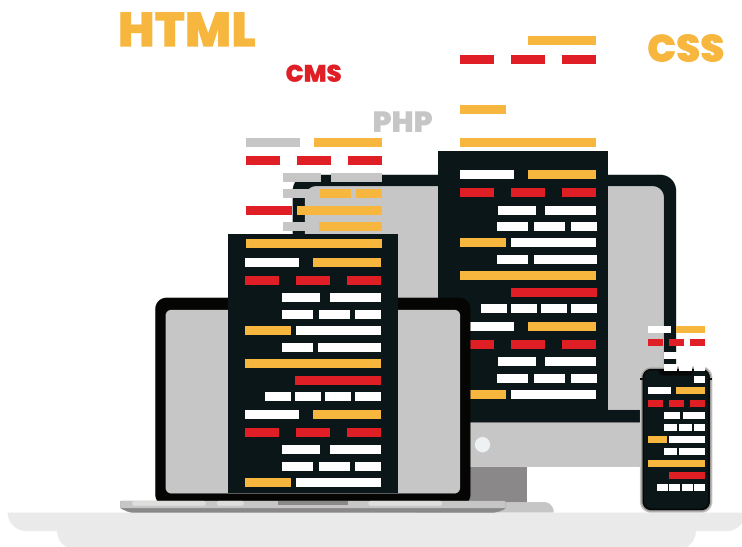
`b=sc.hasNextXYZ()` Trả về true nếu loại còn dữ liệu XYZ để đọc.



## Nhập/xuất tệp tin văn bản

Ví dụ:

```
public static void copyFile(File fromFile,
                           File toFile) throws IOException {
    Scanner freader=new Scanner(fromFile);
    BufferedWriter writer=new BufferedWriter(new FileWriter
    (toFile));
    //...Lặp cho tới khi vẫn còn dòng dữ liệu từ tệp đầu vào.
    String line=null;
    while (freader.hasNextLine()) {
        line=freader.nextLine();
        writer.write(line);
        writer.newLine(); //Viết sang dòng khác trong tệp tin.
    }
    //...Đóng bộ đọc và ghi tệp tin.
    freader.close(); //Đóng để bỏ khóa đối với tệp tin.
    writer.close(); //Đóng để bỏ khóa đối với tệp tin và đẩy toàn
    bộ dữ liệu xuống ổ đĩa.
}
```



## Hướng dẫn đăng ký xét tuyển

**01**

### Đăng ký tư vấn và xét tuyển

Bạn có thể liên hệ CodeGym để nhận được tư vấn và đăng ký xét tuyển bằng một trong các cách sau:

- Gọi đến số hotline: 0246.2538829
- Inbox qua Facebook: [www.fb.com/codegym.vn](https://www.facebook.com/codegym.vn)
- Gửi email đến: [info@codegym.vn](mailto:info@codegym.vn)

**02**

### Nhận lịch hẹn xét tuyển

Sau khi nhận được đăng ký xét tuyển và tư vấn cho bạn khoá học phù hợp, bộ phận tuyển sinh sẽ liên hệ đặt lịch hẹn mời bạn đến tham dự phiên xét tuyển tại các cơ sở đào tạo của CodeGym nơi bạn đăng ký.

**03**

### Thi tuyển và phỏng vấn

Tại phiên xét tuyển, bạn sẽ trải qua các vòng làm bài kiểm tra GMAT và phỏng vấn.

**04**

### Nhận kết quả xét tuyển

Trong vòng 1-2 ngày sau khi tham gia xét tuyển, bộ phận Tuyển sinh sẽ liên hệ thông báo kết quả xét tuyển và hướng dẫn bạn các thủ tục đăng ký nhập học nếu trúng tuyển.

**05**

### Đăng ký nhập học và nộp học phí

Bạn nộp đơn đăng ký nhập học và ký thoả thuận cam kết việc làm.



## CodeGym Hà Nội

**Địa chỉ:** Nhà số 23, Lô TT-01, KĐT Mon City, đường Hàm Nghi, Mỹ Đình 2, Nam Từ Liêm, Hà Nội

**Email:** [info@codegym.vn](mailto:info@codegym.vn)

**Facebook:** [fb.com/codegym.vn](https://fb.com/codegym.vn)

**Website:** [codegym.vn](http://codegym.vn)

**Hotline:** 024 62 538 829

## CodeGym Huế

**Địa chỉ:** Tầng 4, toà B trường Đại học Phú Xuân cơ sở 1, 28 Nguyễn Tri Phương, phường Phú Nhuận, Tp Huế

**Email:** [hue@codegym.vn](mailto:hue@codegym.vn)

**Facebook:** [fb.com/codegymhue](https://fb.com/codegymhue)

**Website:** [hue.codegym.vn](http://hue.codegym.vn)

**Hotline:** 023 46 291 888

## CodeGym Đà Nẵng

**Địa chỉ:** Số 5, Tân Lập 1, phường Thạch Thang, quận Hải Châu, Tp Đà Nẵng.

**Email:** [danang@codegym.vn](mailto:danang@codegym.vn)

**Facebook:** [fb.com/codegymdanang](https://fb.com/codegymdanang)

**Website:** [danang.codegym.vn](http://danang.codegym.vn)

**Hotline:** 023 66 517 021



**Công ty CP CodeGym Việt Nam**